

UNIVERSITY OF PISA

COMPUTER SCIENCE DEPARTMENT

Master's Degree in Data Science & Business Informatics

Comparing Heavy Ball and Bundle Methods for Neural Network Optimization

Candidates:

Niccolò Pratesi (716711)

Abish Jha (716709)

Academic Year 2025/2026

Contents

1 Setting the Stage **3**
1.1 Mathematical Model and Architecture . . . 3
1.2 The Empirical Loss and the Regularized Objective . . . 4
1.3 Theoretical Justification and Design Choices . . . 5
1.3.1 Network Topology: A Shallow Architecture for Bundle Tractability . . . 5
1.3.2 The Choice of Lasso (ℓ_1) Regularization . . . 6
1.3.3 The Magnitude of λ . . . 6
1.4 Analytical Derivation of the Gradients via Backpropagation . . . 7
1.4.1 Gradients for the Output Layer . . . 7
1.4.2 Gradients for the Hidden Layer . . . 8
1.5 Subdifferential Characterization of the Non-Smooth Penalty . . . 8
1.6 Formulation of the Optimization Algorithms . . . 9
1.6.1 The Heavy Ball Method ($A1$) . . . 10
1.6.2 The Proximal Bundle Method ($A2$) . . . 11

2 What to Expect from the Algorithms **13**
2.1 A Non-Convex and Non-Smooth objective function . . . 13
2.1.1 Non-Convexity: . . . 13
2.1.2 Non-Smoothness of the Regularizer: . . . 14
2.1.3 Compactness of Sublevel Sets and Existence of Minima . . . 14
2.2 Expectations for the Subgradient Heavy Ball Method ($A1$) . . . 15
2.2.1 Computational Cost per Iteration. . . . 15
2.2.2 Convergence Results and Theoretical Inapplicability. . . . 16
2.2.3 Trajectory Behavior Around L_1 Kinks. . . . 17
2.2.4 Stopping Criteria and Practical Termination. . . . 17
2.3 Expectations for the Proximal Bundle Method ($A2$) . . . 19
2.3.1 Computational Workload and Master Problem Overhead. . . . 19
2.3.2 Theoretical Convergence Results and Inapplicability. . . . 20
2.3.3 Trajectory Behavior and Kink Stabilization. . . . 21
2.3.4 Stopping Criteria and Certified Optimality Gap. . . . 22
2.4 Analytical Complexity and the Impact of Large Dimension n . . . 22

1 Setting the Stage

Training a machine learning model essentially translates to solving a continuous, unconstrained mathematical optimization problem. In this context, an empirical risk function measuring the discrepancy between model predictions and target observations is minimized alongside a regularization term designed to control model complexity and prevent overfitting. Crucially, pure optimization and machine learning pursue different objectives: finding the exact minimum of a function versus achieving good generalization on unseen data. In our study, training the neural network yields a high-dimensional, non-convex landscape whose computational tractability is largely dictated by the chosen architecture and regularizer.

In this work, we investigate this interplay by focusing on a supervised multi-class classification problem solved through a neural network architecture. To address the resulting optimization problem, we analyze and compare two conceptually distinct algorithmic approaches:

- **The Heavy Ball Method (A1):** A classical accelerated first-order method that introduces an inertial momentum term into the standard gradient descent update [11]. By accumulating past displacement vectors, the algorithm builds resistance to abrupt directional oscillations along high-curvature valleys, substantially speeding up convergence across flat surfaces while maintaining a low computational complexity of $\mathcal{O}(n)$ per iteration.

- **The Bundle Method (A2):** Bundle methods represent a powerful class of algorithms originally devised for non-smooth convex optimization. Rather than relying solely on the local direction at the current point, they build a piecewise-linear lower model (a cutting-plane model) of the objective function by storing previous evaluations of function values and subgradient vectors into a memory bundle.

  In this work, we adopt the classical **Proximal Bundle Method** [7] as our reference implementation. In this formulation, the cutting-plane model is stabilized around the current center via an explicit quadratic proximal term, $\frac{1}{2\tau}\|d\|_2^2$, which keeps the candidate displacement within a region where the approximation remains accurate. As a result, determining the next search direction translates into solving a standard convex Quadratic Programming (QP)[1] Master Problem at every iteration, trading a higher computational cost per step for superior numerical stability and robust handling of non-smooth points.

The remainder of this section is dedicated to formalizing the mathematical groundwork of our benchmark. We first define the exact network topology and its parameter space, proceed with the analytical formulation of the regularized classification loss, and finally provide rigorous theoretical arguments justifying our architectural and algorithmic choices.

1.1 Mathematical Model and Architecture

We consider a supervised multi-class classification problem. Let the training dataset be denoted by:

$$\mathcal{D} = \{(x_p, y_p)\}_{p=1}^{N}, \quad x_p \in \mathbb{R}^d, \quad y_p \in \{0,1\}^K, \tag{1}$$

where N is the total number of training samples, d is the number of input features, and y_p is the one-hot encoded label representing one of K distinct classes ($\sum_{k=1}^{K} y_{pk} = 1$).

[1]Quadratic Programming (QP) refers to the class of continuous optimization problems where a quadratic objective function is minimized subject to linear equality and inequality constraints.

We define a shallow neural network with a single hidden layer comprising m neurons. The trainable parameters of the network consist of two weight matrices and their respective bias vectors:

$$W^{(1)} \in \mathbb{R}^{m\times d}, \qquad b^{(1)} \in \mathbb{R}^{m}, \tag{2}$$

$$W^{(2)} \in \mathbb{R}^{K\times m}, \qquad b^{(2)} \in \mathbb{R}^{K}. \tag{3}$$

We collect all adjustable parameters into a single vector $\Theta \in \mathbb{R}^n$:

$$\Theta = \operatorname{vec}\left(W^{(1)}, b^{(1)}, W^{(2)}, b^{(2)}\right), \tag{4}$$

where the total parameter dimension is given by:

$$n = m \cdot d + m + K \cdot m + K. \tag{5}$$

For an input sample $x_p \in \mathbb{R}^d$, the forward pass through the network is computed as follows:

1. **Hidden layer pre-activation:**

$$z_p^{(1)} = W^{(1)} x_p + b^{(1)} \in \mathbb{R}^m. \tag{6}$$

2. **Hidden layer activation:** We employ the **hyperbolic tangent** (tanh) element-wise activation function. Because tanh is smooth and continuously differentiable, it ensures that any non-differentiability in the objective function arises solely from the ℓ_1 penalty:

$$a_p^{(1)} = \sigma(z_p^{(1)}) = \tanh(z_p^{(1)}) \in \mathbb{R}^m, \quad \sigma(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}}. \tag{7}$$

3. **Linear output layer:** For our optimization-oriented setup, we adopt linear output units. The model prediction $\hat{y}_p \in \mathbb{R}^K$ coincides directly with the output pre-activations:

$$\hat{y}_p(\Theta) = z_p^{(2)} = W^{(2)} a_p^{(1)} + b^{(2)} \in \mathbb{R}^K. \tag{8}$$

At inference time, the discrete class label assigned to pattern x_p is determined via the standard winner-takes-all decision rule:

$$\hat{c}(x_p) = \operatorname{argmax}_{k\in\{1,\dots,K\}} \hat{y}_{pk}(\Theta), \tag{9}$$

while during optimization, the continuous predictions $\hat{y}_p \in \mathbb{R}^K$ are compared directly against the one-hot target vectors $y_p \in \{0,1\}^K$ via the MSE objective.

1.2 The Empirical Loss and the Regularized Objective

To measure how well the neural network fits the training data, we employ the normalized Mean Squared Error (MSE) loss function. For a training set of N samples, let $y_p \in \{0,1\}^K$ denote the one-hot target vector and let $\hat{y}_p(\Theta) \in \mathbb{R}^K$ represent the corresponding network prediction for the input x_p. The empirical loss $\mathcal{L}(\Theta)$ is defined as the average squared Euclidean distance between predictions and ground-truth targets:

$$\mathcal{L}(\Theta) = \frac{1}{N}\sum_{p=1}^{N} \|\hat{y}_p(\Theta) - y_p\|_2^2 = \frac{1}{N}\sum_{p=1}^{N}\sum_{k=1}^{K} \left(\hat{y}_{pk}(\Theta) - y_{pk}\right)^2. \tag{10}$$

The choice of the MSE objective over categorical cross-entropy is particularly well-suited for an optimization-oriented study. From an analytical perspective, the Euclidean quadratic loss yields an explicit, decoupled derivative during backpropagation without numerical instabilities arising from logarithmic evaluations near zero probability values. Furthermore, since the network is constructed via linear combinations and smooth hyperbolic tangent (tanh) activations, the resulting empirical loss is infinitely continuously differentiable, i.e., $\mathcal{L} \in \mathcal{C}^\infty(\mathbb{R}^n)$, ensuring that $\nabla\mathcal{L}(\Theta)$ is globally continuous.

As dictated by the project requirements, a regularization penalty $\Omega(\Theta)$ is incorporated to control model complexity and prevent overfitting. We formulate the training task as an unconstrained scalarized optimization problem:

$$\min_{\Theta\in\mathbb{R}^n} f(\Theta) \coloneqq \mathcal{L}(\Theta) + \lambda\,\Omega(\Theta), \tag{11}$$

where $\lambda > 0$ is a scalar hyperparameter that balances the trade-off between fitting the observed samples and constraining the parameter magnitude (the bias/variance dilemma).

In our formulation, we adopt the ℓ_1-norm penalty (Lasso regularization):

$$\Omega(\Theta) = \|\Theta\|_1 = \sum_{i=1}^{n} |\Theta_i|. \tag{12}$$

Analytically, the penalty $\Omega(\Theta)$ is continuous, convex, and coercive ($\lim_{\|\Theta\|\to\infty} \Omega(\Theta) = +\infty$), but fails to be differentiable wherever at least one parameter vanishes ($\Theta_i = 0$).

Consequently, the full unconstrained optimization problem takes the explicit form:

$$\min_{\Theta\in\mathbb{R}^n} f(\Theta) = \frac{1}{N}\sum_{p=1}^{N}\sum_{k=1}^{K}\left(\hat{y}_{pk}(\Theta) - y_{pk}\right)^2 + \lambda\sum_{i=1}^{n}|\Theta_i|. \tag{13}$$

1.3 Theoretical Justification and Design Choices

The chosen setup reflects specific theoretical and computational considerations directly tied to the two algorithms under analysis: the Heavy Ball method ($A1$) and the Bundle method ($A2$).

1.3.1 Network Topology: A Shallow Architecture for Bundle Tractability

Modern deep learning models often utilize deep architectures with millions of parameters. However, in our setting, the focus is placed on the mathematical behavior of the optimization algorithms rather than maximizing raw classification performance on a benchmark.

We deliberately choose a shallow network, restricting the hidden layer width to a few tens up to roughly one hundred neurons, in order to keep n in the order of hundreds to a few thousand variables.

The main justification for this need lies in the computational complexity of the Bundle method (A2):

- At every iteration, a proximal Bundle method solves a Quadratic Programming Master Problem to aggregate accumulated linear cutting planes and compute the next search direction.
- When working in the primal space, the number of decision variables in this subproblem directly scales with the parameter dimension n. Even in the dual formulation where the

variables correspond to multipliers of the bundle size, operations involving parameter inner products and bundle management scale linearly with n.

- If n were very large (e.g., $10^5 - 10^7$ parameters), the execution time of the QP solver at each iteration would become the dominant bottleneck, rendering the Bundle method computationally impractical. Restricting the parameter space ensures that the Master Problem remains computationally feasible while preserving the non-convex and non-smooth properties of the objective function.

  A detailed analysis of this duality and its computational implications is provided in Section 2.4.

1.3.2 The Choice of Lasso (ℓ_1) Regularization

The choice of the ℓ_1-norm $\Omega(\Theta) = \|\Theta\|_1$ is motivated by optimization theory:

- **Controlled Non-Smoothness:** The smooth loss $\mathcal{L}(\Theta)$ combined with the ℓ_1 norm creates non-differentiable points (kinks) along the coordinate axes where any parameter $\Theta_i = 0$. Since Bundle methods were specifically designed to minimize non-smooth convex functions by collecting subgradient information, this penalty provides an ideal test environment to evaluate the cutting-plane mechanism.

- **Comparison with Momentum:** Standard momentum methods like Heavy Ball assume differentiability (∇f continuous). Near coordinates where $\Theta_i = 0$ a classical gradient does not exist and the subgradient direction can jump discontinuously. This setting allows us to investigate the resilience and stability of Heavy Ball when applied outside its standard smooth framework.

1.3.3 The Magnitude of λ

In accordance with the project guidelines, λ is treated here as a fixed structural parameter defining the mathematical program rather than an empirical hyperparameter to be tuned via cross-validation. To ensure a fair comparison, both algorithms must solve the exact same objective landscape.

We fix $\lambda = 10^{-3}$ based on scale balancing between the two terms:

- While the normalized empirical loss $\mathcal{L}(\Theta)$ is bounded and typically lies in the interval $[0.1, 2.0]$, the Lasso penalty $\|\Theta\|_1 = \sum_{i=1}^{n} |\Theta_i|$ accumulates additively across all $n \approx 10^3$ network parameters.

- Setting λ significantly larger (e.g., $\lambda \geq 10^{-1}$) would cause the regularization term to dwarf the empirical loss ($\lambda\|\Theta\|_1 \gg \mathcal{L}(\Theta)$), driving the step directions entirely by $-\lambda \operatorname{sign}(\Theta)$ and triggering an immediate collapse of all weights to zero.

- Choosing $\lambda = 10^{-3} \approx \mathcal{O}(1/n)$ balances the initial subgradient magnitudes, preventing trivial underfitting while ensuring the ℓ_1-norm introduces the sharp, non-differentiable kinks required to benchmark the cutting-plane mechanism of the Bundle method against Heavy Ball.

1.4 Analytical Derivation of the Gradients via Backpropagation

To minimize the composite objective function without resorting to automatic differentiation tools, we must explicitly compute its derivatives by hand. Since the objective decomposes into a smooth part and a non-smooth part as $f(\Theta) = \mathcal{L}(\Theta) + \lambda\Omega(\Theta)$, we first focus on the gradient of the empirical loss $\nabla\mathcal{L}(\Theta)$ by deriving the backpropagation equations step-by-step using the chain rule.

Recall that the total empirical loss is the average over all N independent training pairs:

$$\mathcal{L}(\Theta) = \frac{1}{N}\sum_{p=1}^{N}\mathcal{L}_p(\Theta), \quad \text{where} \quad \mathcal{L}_p(\Theta) = \sum_{k=1}^{K}\left(\hat{y}_{pk}(\Theta) - y_{pk}\right)^2. \tag{14}$$

By linearity of the differentiation operator, it suffices to compute the partial derivatives for a single pattern p and average them over the dataset:

$$\frac{\partial\mathcal{L}}{\partial\Theta} = \frac{1}{N}\sum_{p=1}^{N}\frac{\partial\mathcal{L}_p}{\partial\Theta}. \tag{15}$$

1.4.1 Gradients for the Output Layer

For the output layer, the pre-activation vector is $z_p^{(2)} = W^{(2)}a_p^{(1)} + b^{(2)}$. We define the output local error vector $\delta_p^{(2)} \in \mathbb{R}^K$ as the sensitivity of the loss with respect to the output pre-activations:

$$\delta_{pk}^{(2)} := \frac{\partial\mathcal{L}_p}{\partial z_{pk}^{(2)}}. \tag{16}$$

Applying the standard chain rule through the prediction $\hat{y}_{pk}$:

$$\delta_{pk}^{(2)} = \frac{\partial\mathcal{L}_p}{\partial\hat{y}_{pk}} \cdot \frac{\partial\hat{y}_{pk}}{\partial z_{pk}^{(2)}}. \tag{17}$$

For linear output units ($\hat{y}_{pk} = z_{pk}^{(2)}$), the derivative is $\frac{\partial\hat{y}_{pk}}{\partial z_{pk}^{(2)}} = 1$. Differentiating the quadratic error term yields:

$$\frac{\partial\mathcal{L}_p}{\partial\hat{y}_{pk}} = 2\left(\hat{y}_{pk} - y_{pk}\right) \implies \delta_{pk}^{(2)} = 2\left(\hat{y}_{pk} - y_{pk}\right). \tag{18}$$

In compact vector notation:

$$\delta_p^{(2)} = 2\left(\hat{y}_p - y_p\right) \in \mathbb{R}^K. \tag{19}$$

Using $\delta_p^{(2)}$, we apply the chain rule to compute the partial derivatives with respect to the output parameters $W^{(2)} \in \mathbb{R}^{K\times m}$ and $b^{(2)} \in \mathbb{R}^K$:

$$\frac{\partial\mathcal{L}_p}{\partial W_{kj}^{(2)}} = \frac{\partial\mathcal{L}_p}{\partial z_{pk}^{(2)}}\frac{\partial z_{pk}^{(2)}}{\partial W_{kj}^{(2)}} = \delta_{pk}^{(2)}\, a_{pj}^{(1)}, \tag{20}$$

$$\frac{\partial\mathcal{L}_p}{\partial b_k^{(2)}} = \frac{\partial\mathcal{L}_p}{\partial z_{pk}^{(2)}}\frac{\partial z_{pk}^{(2)}}{\partial b_k^{(2)}} = \delta_{pk}^{(2)}. \tag{21}$$

Expressing these relations via outer products across all units:

$$\frac{\partial\mathcal{L}_p}{\partial W^{(2)}} = \delta_p^{(2)}\left(a_p^{(1)}\right)^T \in \mathbb{R}^{K\times m}, \qquad \frac{\partial\mathcal{L}_p}{\partial b^{(2)}} = \delta_p^{(2)} \in \mathbb{R}^K. \tag{22}$$

1.4.2 Gradients for the Hidden Layer

To propagate the error backwards to the hidden layer, we compute the sensitivity vector $\delta_p^{(1)} \in \mathbb{R}^m$ defined as:

$$\delta_{pj}^{(1)} \coloneqq \frac{\partial \mathcal{L}_p}{\partial z_{pj}^{(1)}} = \frac{\partial \mathcal{L}_p}{\partial a_{pj}^{(1)}} \cdot \frac{\partial a_{pj}^{(1)}}{\partial z_{pj}^{(1)}}. \tag{23}$$

The activation $a_{pj}^{(1)}$ influences the loss through all K output units. Therefore, applying the multivariate chain rule:

$$\frac{\partial \mathcal{L}_p}{\partial a_{pj}^{(1)}} = \sum_{k=1}^{K} \frac{\partial \mathcal{L}_p}{\partial z_{pk}^{(2)}} \frac{\partial z_{pk}^{(2)}}{\partial a_{pj}^{(1)}} = \sum_{k=1}^{K} \delta_{pk}^{(2)} W_{kj}^{(2)} = \left(\left(W^{(2)} \right)^T \delta_p^{(2)} \right)_j. \tag{24}$$

Next, we evaluate the derivative of the activation function $\sigma(t) = \tanh(t)$. The derivative of the hyperbolic tangent function satisfies:

$$\sigma'(t) = 1 - \tanh^2(t) = 1 - \sigma(t)^2. \tag{25}$$

Consequently, the local sensitivity for the hidden unit j is:

$$\delta_{pj}^{(1)} = \left(\sum_{k=1}^{K} \delta_{pk}^{(2)} W_{kj}^{(2)} \right) \cdot \left(1 - \left(a_{pj}^{(1)} \right)^2 \right). \tag{26}$$

In vector notation, letting $\odot$ denote the Hadamard (element-wise) product:

$$\delta_p^{(1)} = \left(\left(W^{(2)} \right)^T \delta_p^{(2)} \right) \odot \left(\mathbf{1} - a_p^{(1)} \odot a_p^{(1)} \right) \in \mathbb{R}^m. \tag{27}$$

Finally, applying the chain rule with respect to the input weights $W^{(1)} \in \mathbb{R}^{m \times d}$ and biases $b^{(1)} \in \mathbb{R}^m$:

$$\frac{\partial \mathcal{L}_p}{\partial W_{ji}^{(1)}} = \frac{\partial \mathcal{L}_p}{\partial z_{pj}^{(1)}} \frac{\partial z_{pj}^{(1)}}{\partial W_{ji}^{(1)}} = \delta_{pj}^{(1)} \, x_{pi}, \tag{28}$$

$$\frac{\partial \mathcal{L}_p}{\partial b_j^{(1)}} = \frac{\partial \mathcal{L}_p}{\partial z_{pj}^{(1)}} \frac{\partial z_{pj}^{(1)}}{\partial b_j^{(1)}} = \delta_{pj}^{(1)}. \tag{29}$$

In matrix form:

$$\frac{\partial \mathcal{L}_p}{\partial W^{(1)}} = \delta_p^{(1)} x_p^T \in \mathbb{R}^{m \times d}, \qquad \frac{\partial \mathcal{L}_p}{\partial b^{(1)}} = \delta_p^{(1)} \in \mathbb{R}^m. \tag{30}$$

Averaging across all N patterns yields the complete analytical gradient $\nabla \mathcal{L}(\Theta)$.

1.5 Subdifferential Characterization of the Non-Smooth Penalty

The regularization term $\Omega(\Theta) = \|\Theta\|_1 = \sum_{i=1}^{n} |\Theta_i|$ is continuous and convex, but it fails to be differentiable whenever any coordinate vanishes ($\Theta_i = 0$). In convex analysis, generalized first-order information is captured by the subdifferential [6].

Definition 1. *Let $h : \mathbb{R}^n \to \mathbb{R}$ be a convex function. The subdifferential of h at Θ, denoted by $\partial h(\Theta)$, is the set of all vectors $g \in \mathbb{R}^n$ satisfying the following inequality:*

$$h(w) \geq h(\Theta) + \langle g, w - \Theta \rangle, \quad \forall w \in \mathbb{R}^n. \tag{31}$$

Since the ℓ_1-norm is completely separable across coordinates, its subdifferential is the Cartesian product of the scalar subdifferentials of the absolute value function [6]:

$$\partial\|\Theta\|_1 = \partial|\Theta_1| \times \partial|\Theta_2| \times \cdots \times \partial|\Theta_n|. \tag{32}$$

Proposition 1. *The subdifferential of the univariate absolute value function $h(t) = |t|$ is given by:*

$$\partial|t| = \begin{cases} \{1\} & \text{if } t > 0, \\ \{-1\} & \text{if } t < 0, \\ [-1, 1] & \text{if } t = 0. \end{cases} \tag{33}$$

Proof. We analyze the three cases directly from the definition:

1. If $t > 0$, the function is smooth with classical derivative $\frac{d}{dt}(t) = 1$. Hence, $\partial|t| = \{1\}$.
2. If $t < 0$, the function is smooth with classical derivative $\frac{d}{dt}(-t) = -1$. Hence, $\partial|t| = \{-1\}$.
3. If $t = 0$, a scalar $s \in \mathbb{R}$ is a subgradient if and only if:

   $$|w| \geq |0| + s(w - 0) \iff |w| \geq s \cdot w, \quad \forall w \in \mathbb{R}.$$

   Evaluating this condition for non-zero values of w:

   - For $w > 0$: dividing by w gives $s \leq \frac{w}{w} = 1$.
   - For $w < 0$: dividing by w (reversing the inequality sign) gives $s \geq \frac{-w}{w} = -1$.

   Combining these two bounds yields $-1 \leq s \leq 1$. Hence, the set of all valid subgradients at the origin is the closed interval $[-1, 1]$.

□

By Moreau-Rockafellar's subdifferential calculus rule for the sum of a differentiable function and a convex function [6], the subdifferential of the complete objective function $f(\Theta) = \mathcal{L}(\Theta) + \lambda\|\Theta\|_1$ is:

$$\partial f(\Theta) = \nabla\mathcal{L}(\Theta) + \lambda\, \partial\|\Theta\|_1. \tag{34}$$

In practical algorithmic implementations, evaluating a concrete subgradient vector $g(\Theta) \in \partial f(\Theta)$ requires selecting a specific element from the set $[-1, 1]$ whenever $\Theta_i = 0$. We adopt the standard canonical choice $0 \in [-1, 1]$, which yields the explicit component-wise formula:

$$g_i(\Theta) = \nabla\mathcal{L}(\Theta)_i + \lambda\ \text{sign}(\Theta_i), \quad \text{where} \quad \text{sign}(t) = \begin{cases} 1 & \text{if } t > 0, \\ -1 & \text{if } t < 0, \\ 0 & \text{if } t = 0. \end{cases} \tag{35}$$

1.6 Formulation of the Optimization Algorithms

Having established the mathematical model of the neural network and the Lasso-regularized objective function, we now formalize the two optimization schemes under comparison.

1.6.1 The Heavy Ball Method ($A1$)

The Heavy Ball algorithm is a two-step first-order method where at each iteration t, the displacement combines the current descent direction with an inertial momentum component proportional to the displacement of the preceding iteration [11]:

$$p_t = -g_t + \beta\, p_{t-1}, \tag{36}$$

$$\Theta_{t+1} = \Theta_t + \alpha\, p_t, \tag{37}$$

where $\alpha > 0$ is the step-size parameter, $\beta \in [0, 1)$ is the momentum coefficient and $p_0 = 0$. Equivalently, substituting (36) into (37), the recurrence can be stated in its classical three-term form:

$$\Theta_{t+1} = \Theta_t - \alpha\, g_t + \beta(\Theta_t - \Theta_{t-1}). \tag{38}$$

Subgradient Extension under Non-Smooth Regularization: Because the ℓ_1-norm penalty introduces nondifferentiability wherever parameters vanish ($\Theta_i = 0$), the classical gradient $\nabla f(\Theta_t)$ is not defined globally. In our implementation, we adapt the method by substituting the gradient with the canonical subgradient vector $g_t = g(\Theta_t) \in \partial f(\Theta_t)$ defined in (35). Each component evaluates to:

$$(g_t)_i = \nabla \mathcal{L}(\Theta_t)_i + \lambda\, s_i, \quad \text{where } s_i = \begin{cases} \operatorname{sign}(\Theta_{t,i}) & \text{if } \Theta_{t,i} \neq 0, \\ 0 & \text{if } \Theta_{t,i} = 0, \end{cases} \tag{39}$$

which corresponds to selecting the canonical element $0 \in [-1, 1] = \partial|0|$.

Theoretical Limitations and Algorithmic Discrepancy: While this direct subgradient substitution is widely adopted in machine learning practice, it represents an essentially heuristic extension. Classical convergence proofs for the Heavy Ball method require the objective function to be continuously differentiable and convex (or satisfy local growth conditions) to ensure that the momentum term does not cause divergence [11, 10]. In our setting, $f(\Theta)$ is simultaneously *non-convex* (due to the nonlinear neural architecture) and *non-smooth* (due to the ℓ_1-norm kinks). In the presence of kinks, $-g_t$ is not necessarily a direction of descent and when coupled with the accumulated velocity βp_{t-1}, the trajectory can undergo substantial overshoot and high-frequency oscillations across non-differentiable boundaries.

To properly handle such composite problems with momentum, literature suggests methods like iPiano [10], which apply an inertial gradient step to the smooth loss $\mathcal{L}(\Theta)$ and treat the non-smooth term $\lambda\|\Theta\|_1$ separately with a soft-thresholding operator. In this work, we keep the subgradient Heavy Ball as a baseline to test whether classical momentum remains effective on non-smooth problems, contrasting its behavior with the Proximal Bundle Method.

Stopping Criteria: In classical unconstrained optimization, the theoretical termination rule for the Heavy Ball method relies on identifying a stationary point where the first-order necessary optimality condition is satisfied. The execution terminates when either of the following two conditions is met:

- **First-Order Optimality Certificate:** The Euclidean norm of the gradient drops below a prescribed tolerance threshold $\varepsilon > 0$:

$$\|\nabla f(\Theta_t)\|_2 \leq \varepsilon, \tag{40}$$

or, in scale-invariant relative form:

$$\frac{\|\nabla f(\Theta_t)\|_2}{\max\{1, \|\nabla f(\Theta_0)\|_2\}} \leq \varepsilon. \tag{41}$$

The gradient norm serves as an operational proxy for optimality, indicating that the local linear approximation is flat and certifying that the iterate is in the immediate vicinity of a stationary point.

- **Maximum Iterations Budget:** To guarantee finite termination in practical implementations, the algorithm stops if the iteration counter reaches the allocated budget:

$$t \geq t_{\max}. \tag{42}$$

1.6.2 The Proximal Bundle Method ($A2$)

The Proximal Bundle method builds a cutting-plane model from past evaluations and stabilizes it with a quadratic proximal term:

Suppose that at iteration k, the algorithm maintains a current stability center $\bar{\Theta}$ and a bundle of past information collected at points Θ_j for $j \in \mathcal{B}$:

$$\mathcal{B} = \{(\Theta_j, f(\Theta_j), g_j) : g_j \in \partial f(\Theta_j)\}. \tag{43}$$

For each evaluated point Θ_j, the subgradient inequality defines a global affine lower bounding plane (cutting plane):

$$f(\Theta) \geq f(\Theta_j) + \langle g_j, \Theta - \Theta_j \rangle = f(\bar{\Theta}) + \langle g_j, \Theta - \bar{\Theta} \rangle - \alpha_j, \tag{44}$$

where $\alpha_j \geq 0$ is the linearization error measured relative to the stability center $\bar{\Theta}$:

$$\alpha_j \coloneqq f(\bar{\Theta}) - f(\Theta_j) - \langle g_j, \bar{\Theta} - \Theta_j \rangle. \tag{45}$$

The bundle defines a piecewise-affine lower cutting-plane model $\check{f}(\Theta)$:

$$\check{f}(\bar{\Theta} + d) = \max_{j \in \mathcal{B}} \left\{ f(\bar{\Theta}) + \langle g_j, d \rangle - \alpha_j \right\}. \tag{46}$$

To prevent the model from making excessively large steps into regions where the cutting-plane approximation is inaccurate, a quadratic proximal stabilization term $\frac{1}{2\tau}\|d\|_2^2$ is added, where $\tau > 0$ acts as a proximity parameter (analogous to an inverse trust-region radius).

The candidate displacement $d \in \mathbb{R}^n$ is computed by solving the **Primal Master Problem**:

$$\min_{d \in \mathbb{R}^n,\, v \in \mathbb{R}} \left\{ v + \frac{1}{2\tau}\|d\|_2^2 \quad : \quad v \geq \langle g_j, d \rangle - \alpha_j, \quad \forall j \in \mathcal{B} \right\}, \tag{47}$$

where v represents the predicted change in objective value. Problem (47) is a strictly convex Quadratic Program with $n + 1$ variables and $|\mathcal{B}|$ linear inequality constraints.

To eliminate the parameter dimension n from the optimization variables, we derive its dual. Associating Lagrange multipliers $\mu_j \geq 0$ with each cutting plane constraint:

$$\mathcal{L}(d, v, \mu) = v + \frac{1}{2\tau}\|d\|_2^2 + \sum_{j \in \mathcal{B}} \mu_j \left(\langle g_j, d \rangle - \alpha_j - v \right). \tag{48}$$

Setting the partial derivatives with respect to the primal variables to zero:

$$\nabla_v \mathcal{L} = 0 \implies \sum_{j \in \mathcal{B}} \mu_j = 1, \tag{49}$$

$$\nabla_d \mathcal{L} = 0 \implies \frac{1}{\tau} d + \sum_{j \in \mathcal{B}} \mu_j g_j = 0 \implies d(\mu) = -\tau \sum_{j \in \mathcal{B}} \mu_j g_j. \tag{50}$$

Substituting these optimality conditions back into the Lagrangian yields the **Dual Master Problem**:

$$\min_{\mu \in \mathbb{R}^{|\mathcal{B}|}} \left\{ \frac{\tau}{2} \left\| \sum_{j \in \mathcal{B}} \mu_j g_j \right\|_2^2 + \sum_{j \in \mathcal{B}} \mu_j \alpha_j \quad : \quad \sum_{j \in \mathcal{B}} \mu_j = 1, \quad \mu_j \geq 0, \ \forall j \in \mathcal{B} \right\}. \tag{51}$$

Problem (51) is a standard convex QP over the unitary simplex $\Delta_{|\mathcal{B}|}$. The decision variables are strictly limited to the size of the bundle $|\mathcal{B}| \ll n$.

Once the optimal dual multipliers μ^* are obtained using an off-the-shelf QP solver, the candidate displacement d^* is recovered analytically via (50). The function value and subgradient are then evaluated at the trial point $\Theta^+ = \bar{\Theta} + d^*$:

- **Serious Step:** If the actual reduction satisfies $f(\bar{\Theta}) - f(\Theta^+) \geq m_L \left(-v^*\right)$ (for a fixed parameter $m_L \in (0, 1)$), the step is declared serious: the stability center is updated ($\bar{\Theta} \leftarrow \Theta^+$), and the new point is added to the bundle.

- **Null Step:** If the reduction is insufficient, the center remains unchanged ($\bar{\Theta} \leftarrow \bar{\Theta}$), but the newly computed information $(\Theta^+, f(\Theta^+), g^+)$ is appended to the bundle $\mathcal{B}$, refining the cutting-plane model for subsequent QP solves.

This step classification also dictates how we manage the proximity parameter $\tau > 0$. Geometrically, τ acts as a **trust region radius** around the current stability center: a **large** τ weakens the quadratic penalty $\frac{1}{2\tau}\|d\|_2^2$, allowing **longer exploratory steps** based almost entirely on the shape of the cutting planes, whereas a **small** τ creates a steep quadratic penalty, acting like a **gravitational anchor** that keeps the candidate displacement d^* very close to $\bar{\Theta}$.

In optimization practice, there are two main ways to handle this parameter:

- **Fixed Parameter Setting:** One simple option is to keep τ constant across all iterations. However, this choice has clear limitations: if τ is too large, the algorithm keeps taking overly ambitious steps where the linear approximation fails; if it is too small, progress becomes extremely slow because the steps are forced to stay tiny.

- **Adaptive Trust-Region Heuristic:** To overcome these drawbacks, it is generally much better to update τ dynamically based on how well the model predicts the actual objective decrease:

  - After a **Serious Step**, the linear model proves to be reliable, so τ is enlarged to encourage **larger steps** using an expansion factor $\gamma_{\text{inc}} > 1$ (typically bounded by a maximum threshold $\tau_{\max}$):

$$\tau \leftarrow \min\left\{\gamma_{\text{inc}}\, \tau, \, \tau_{\max}\right\}; \tag{52}$$

– After repeated **Null Steps**, the cutting planes overestimate the true descent, showing that the trial point went too far; τ is therefore reduced by a contraction factor $\gamma_{\text{dec}} \in (0, 1)$ (bounded from below by $\tau_{\min} > 0$) to **shrink the search radius** back toward the stability center until new cuts restore accuracy:

$$\tau \leftarrow \max\left\{\gamma_{\text{dec}}\,\tau,\ \tau_{\min}\right\}. \tag{53}$$

Stopping Criteria: The iterative execution terminates when either of the following conditions is satisfied:

- **Maximum Iterations Budget:** The iteration counter reaches the computational budget $k \geq k_{\max}$;
- **Model Optimality Certificate:** The predicted decrease obtained from the Master Problem drops below a given tolerance threshold ε, indicating that the stability center is close to an approximate stationary point:

$$-v^* \leq \epsilon \iff \tau\|g_{\text{agg}}\|_2^2 + \sum_{j\in\mathcal{B}} \mu_j^* \alpha_j \leq \epsilon. \tag{54}$$

2 What to Expect from the Algorithms

Before running the numerical experiments, it is essential to analyze the theoretical properties of our problem and compare them with the assumptions required by the convergence theorems of the chosen algorithms. This analysis highlights a clear gap between textbook theory and our practical setting.

2.1 A Non-Convex and Non-Smooth objective function

Our goal is to minimize the composite objective function:

$$f(\Theta) = \mathcal{L}(\Theta) + \lambda\|\Theta\|_1. \tag{55}$$

which is simultaneously **non-convex** and **non-smooth**, as shown in the previous chapter.

2.1.1 Non-Convexity:

Recall from convex analysis that a twice continuously differentiable function $\mathcal{L} \in \mathcal{C}^2(\mathbb{R}^n)$ is convex if and only if its Hessian matrix is positive semi-definite everywhere:

$$\mathcal{L} \text{ is convex on } \mathbb{R}^n \iff \nabla^2\mathcal{L}(\Theta) \succeq 0, \quad \forall\Theta \in \mathbb{R}^n, \tag{56}$$

meaning that for every parameter vector Θ and any non-zero direction $d \in \mathbb{R}^n$, the directional curvature satisfies $d^T\nabla^2\mathcal{L}(\Theta)d \geq 0$.

In the case of our neural network trained with the MSE, this condition fails. To understand why $\nabla^2\mathcal{L}(\Theta)$ is not positive semi-definite everywhere, we examine the analytical structure of the Hessian of the empirical loss $\mathcal{L}(\Theta) = \frac{1}{N}\sum_{p=1}^{N}\sum_{k=1}^{K}(\hat{y}_{pk}(\Theta) - y_{pk})^2$:

$$\nabla^2\mathcal{L}(\Theta) = \frac{2}{N}\sum_{p=1}^{N}\sum_{k=1}^{K}\left[\nabla\hat{y}_{pk}(\Theta)\nabla\hat{y}_{pk}(\Theta)^T + (\hat{y}_{pk}(\Theta) - y_{pk})\nabla^2\hat{y}_{pk}(\Theta)\right]. \tag{57}$$

The first summation term is known as the Gauss-Newton matrix, which is inherently always positive semi-definite ($\succeq 0$) [8]. However, the second term is weighted by the prediction errors (residuals) $(\hat{y}_{pk} - y_{pk})$ and depends directly on the second derivatives $\nabla^2 \hat{y}_{pk}(\Theta)$ of the network output.

Because the hidden layer employs the hyperbolic tangent activation function, its second derivative is:

$$\frac{d^2}{dt^2} \tanh(t) = -2 \tanh(t) \left(1 - \tanh^2(t)\right), \tag{58}$$

which is non-zero and changes sign depending on whether the pre-activation is positive or negative. In the second term of (57), whenever the prediction error has the opposite sign of the network curvature:

$$(\hat{y}_{pk}(\Theta) - y_{pk}) \nabla^2 \hat{y}_{pk}(\Theta) < 0, \tag{59}$$

this contribution becomes negative. Hence, there exist directions d such that $d^T \nabla^2 \mathcal{L}(\Theta) d < 0$, which directly violates (56) and proves that $\mathcal{L}(\Theta)$ is non-convex.

2.1.2 Non-Smoothness of the Regularizer:

In addition to the non-convex empirical loss, the Lasso regularizer $\lambda \|\Theta\|_1 = \lambda \sum_{i=1}^{n} |\Theta_i|$ introduces non-differentiable points along the coordinate hyperplanes whenever any weight or bias is zero ($\Theta_i = 0$). At these points, classical derivatives cease to exist, requiring the use of generalized subgradient mappings.

2.1.3 Compactness of Sublevel Sets and Existence of Minima

Before analyzing the convergence properties of specific algorithms, it is essential to ensure that the unconstrained optimization problem is mathematically well-posed and that a global minimizer actually exists.

Continuity of the Composite Objective. We first verify that $f(\Theta) = \mathcal{L}(\Theta) + \lambda \|\Theta\|_1$ is continuous on $\mathbb{R}^n$. The loss $\mathcal{L}(\Theta)$ is smooth ($\mathcal{L} \in \mathcal{C}^\infty$) because it is obtained by composing linear combinations of the parameters, the smooth activation tanh and the quadratic MSE loss. The regularizer $\|\Theta\|_1 = \sum_{i=1}^{n} |\Theta_i|$ is a sum of continuous absolute value functions, and is therefore continuous on $\mathbb{R}^n$. Since the sum of continuous functions is continuous, the overall objective satisfies:

$$f \in \mathcal{C}^0(\mathbb{R}^n). \tag{60}$$

Coercivity and Compactness. While $\mathcal{L}(\Theta) \geq 0$ is bounded from below, it is not coercive on its own. However, introducing the Lasso regularizer with $\lambda > 0$ guarantees that $f(\Theta) = \mathcal{L}(\Theta) + \lambda \|\Theta\|_1$ is coercive[2]:

$$\lim_{\|\Theta\| \to \infty} f(\Theta) \geq \lim_{\|\Theta\| \to \infty} \lambda \|\Theta\|_1 = +\infty. \tag{61}$$

Consequently, for any initial point $\Theta_0 \in \mathbb{R}^n$, the sublevel set:

$$\mathcal{S}_0 := \{\Theta \in \mathbb{R}^n : f(\Theta) \leq f(\Theta_0)\} \tag{62}$$

is bounded. Since f is continuous, $\mathcal{S}_0$ is closed and thus compact by the Heine-Borel theorem [12, Theorem 2.41, p. 40]. This compactness ensures two fundamental results:

[2] A function $f : \mathbb{R}^n \to \mathbb{R}$ is coercive if $\lim_{\|\Theta\| \to \infty} f(\Theta) = +\infty$.

- **Existence of Minima (Weierstrass):** Since $f \in \mathcal{C}^0$ on the compact set $\mathcal{S}_0$, f achieves its infimum on $\mathcal{S}_0$ [12, Theorem 4.16, p. 89], guaranteeing the existence of a global minimizer $\Theta^* \in \mathcal{S}_0$.
- **Existence of Accumulation Points (Bolzano-Weierstrass):** For any descent method where $f(\Theta_{t+1}) \leq f(\Theta_t)$ (such as serious steps in Bundle methods), iterates remain confined in $\mathcal{S}_0$. By Bolzano-Weierstrass [12, Theorem 2.42, p. 40], the bounded sequence $\{\Theta_t\}$ admits at least one convergent subsequence, ensuring non-empty cluster points[3] for stationarity analysis.

In summary, because our composite objective simultaneously violates convexity (due to the neural activations) and global differentiability (due to the ℓ_1 kinks), standard convergence guarantees cannot be applied directly to either algorithm. In the following subsections, we analyze in detail what behavior and theoretical discrepancies to expect from the Subgradient Heavy Ball ($A1$) and the Proximal Bundle Method ($A2$).

2.2 Expectations for the Subgradient Heavy Ball Method ($A1$)

Based on its mathematical formulation and the analytical properties of our objective function $f(\Theta) = \mathcal{L}(\Theta) + \lambda\|\Theta\|_1$, we formulate four key expectations regarding its **computational scalability, convergence theory, trajectory dynamics, and stopping behavior.**

2.2.1 Computational Cost per Iteration.

The computational effort required by each iteration t of the Heavy Ball method decomposes additively into two distinct components: the evaluation of the objective function and its subgradient and the algorithmic parameter update:

$$\mathcal{C}_{\text{iteration}} = \mathcal{C}_{\text{eval}} + \mathcal{C}_{\text{update}}. \tag{63}$$

1. **Objective and Subgradient Evaluation ($\mathcal{C}_{\text{eval}}$):** Because optimization is carried out in deterministic full-batch mode, evaluating the loss and its gradient requires processing all N training samples. For each individual sample, both the forward propagation through the layers and the backward propagation sweep scale linearly with the total number of network parameters n. Summing these operations over the entire dataset yields an evaluation cost of:

   $$\mathcal{C}_{\text{eval}} = \mathcal{O}(N \cdot n). \tag{64}$$

   Evaluating the Lasso penalty $\lambda\|\Theta\|_1$ and selecting its subgradient coordinate-wise costs only $\mathcal{O}(n)$, which is completely negligible compared to the full dataset pass.

2. **Algorithmic Parameter Update ($\mathcal{C}_{\text{update}}$):** Once the subgradient vector g_t is computed, the Heavy Ball update executes its two-step recurrence:

   $$p_t = -g_t + \beta p_{t-1}, \tag{65}$$
   $$\Theta_{t+1} = \Theta_t + \alpha p_t. \tag{66}$$

   These steps involve only basic vector additions and scalar multiplications in $\mathbb{R}^n$. The algorithmic overhead is therefore strictly:

   $$\mathcal{C}_{\text{update}} = \mathcal{O}(n). \tag{67}$$

[3] A point $\bar{\Theta}$ is a cluster point of $\{\Theta_t\}$ if a subsequence converges to it: $\lim_{j\to\infty} \Theta_{t_j} = \bar{\Theta}$.

Combining these two terms yields a total time complexity per iteration of:

$$\mathcal{C}_{\text{iteration}} = \mathcal{O}(N \cdot n) + \mathcal{O}(n) = \mathcal{O}(N \cdot n). \tag{68}$$

Similarly, the memory footprint scales as $\mathcal{O}(n)$, since the algorithm only needs to store the current parameter vector Θ_t, the velocity vector p_{t-1}, and the subgradient g_t. The computational runtime per step is therefore completely dominated by the backpropagation pass over the dataset, while the update itself is virtually instantaneous.

2.2.2 Convergence Results and Theoretical Inapplicability.

To understand what convergence behavior we can realistically expect from the Heavy Ball method, we examine the classical results available in optimization literature:

- **Classical Results in the Convex Case:** The original convergence theorem by Polyak [11] proves asymptotic linear convergence under two main assumptions:

  (i) *Smoothness:* The objective function is twice continuously differentiable ($f \in \mathcal{C}^2$) with an L-Lipschitz continuous gradient, meaning that the maximum eigenvalue of the Hessian is bounded: $\nabla^2 f(\Theta) \preceq LI$.

  (ii) *Strong Convexity:* The objective has a positive minimum curvature $\tau > 0$, meaning that the minimum eigenvalue of the Hessian is strictly positive: $\nabla^2 f(\Theta) \succeq \tau I \succ 0$.

  When these conditions hold, choosing the optimal parameters:

  $$\alpha^* = \frac{4}{(\sqrt{L} + \sqrt{\tau})^2}, \qquad \beta^* = \left(\frac{\sqrt{L} - \sqrt{\tau}}{\sqrt{L} + \sqrt{\tau}}\right)^2, \tag{69}$$

  guarantees a fast linear convergence rate governed by the condition number $\kappa = L/\tau$. Later, Ghadimi et al. [4] extended this analysis to general convex functions where the strong convexity is lost ($\tau = 0$), proving a slower, sublinear convergence rate of $\mathcal{O}(1/t)$.

- **The Closest Result in the Non-Convex Setting:** Since our neural network loss is not convex, the closest theoretical result available for the Heavy Ball method is provided by Ochs [9, Theorem 4.1, p. 168]. This theorem shows that for a general non-convex function with an L-Lipschitz gradient ($f \in \mathcal{C}^{1,1}$), the Heavy Ball trajectory converges to a stationary point if $\beta \in [0, 1)$ and the step-size satisfies:

  $$\alpha \in \left(0, \frac{2(1-\beta)}{L}\right). \tag{70}$$

- **Theoretical Inapplicability:** Because our objective violates both convexity (due to the tanh activations) and differentiability (due to the ℓ_1 kinks), none of the classical convergence theorems (Polyak, Ghadimi, Ochs) formally apply. Using the Heavy Ball method with an operational subgradient is therefore a purely heuristic approach.

  Nonetheless, the condition in (70) highlights a practical trade-off: since the bound $2(1 - \beta)/L$ goes to zero as $\beta \to 1$, choosing a larger momentum β naturally requires using a smaller step-size α to keep the updates under control and prevent the algorithm from diverging.

2.2.3 Trajectory Behavior Around L_1 Kinks.

A key issue when running the Heavy Ball method on $f(\Theta) = \mathcal{L}(\Theta) + \lambda\|\Theta\|_1$ is the interaction between momentum and the non-differentiable kinks ($\Theta_i = 0$).

Using the canonical selection $s_i = \text{sign}(\Theta_{t,i})$ for the regularizer subgradient $(g_t)_i = \nabla\mathcal{L}(\Theta_t)_i + \lambda s_i$, the coordinate-wise update reads:

$$\Theta_{t+1,i} = \Theta_{t,i} - \alpha(g_t)_i + \beta(\Theta_{t,i} - \Theta_{t-1,i}). \tag{71}$$

While in smooth optimization approaching a local minimum implies that the gradient vanishes ($\nabla f(\Theta_t) \to 0$), the Lasso subgradient experiences a jump of magnitude 2λ across the origin:

$$\lim_{\Theta_i \to 0^+} g_i(\Theta) - \lim_{\Theta_i \to 0^-} g_i(\Theta) = (\nabla\mathcal{L}_i + \lambda) - (\nabla\mathcal{L}_i - \lambda) = 2\lambda. \tag{72}$$

When an iterate approaches zero, the velocity $\beta p_{t-1,i} \neq 0$ carries the parameter across the axis by inertia. Immediately past the kink, the subgradient flips sign, delivering an abrupt counter-thrust of magnitude $\alpha\lambda$ and causing persistent sign alternations: $\text{sign}(\Theta_{t+1,i}) = -\text{sign}(\Theta_{t,i})$.

To quantify this behavior near an optimal sparse coordinate where the data loss gradient is negligible relative to the regularizer ($|\nabla\mathcal{L}(\Theta_t)_i| \ll \lambda$), we consider a symmetric oscillation of amplitude δ around zero:

$$\Theta_{t-1,i} = +\delta, \quad \Theta_{t,i} = -\delta, \quad \Theta_{t+1,i} = +\delta, \tag{73}$$

with $\delta > 0$. At step t, because $\Theta_{t,i} = -\delta < 0$, the subgradient is dominated by the regularizer sign:

$$(g_t)_i = \nabla\mathcal{L}(\Theta_t)_i + \lambda\,\text{sign}(-\delta) \approx -\lambda. \tag{74}$$

Substituting these terms into the recurrence (71):

$$\delta = -\delta - \alpha(-\lambda) + \beta\big(-\delta - \delta\big) \tag{75}$$

$$\delta = -\delta + \alpha\lambda - 2\beta\delta \tag{76}$$

$$2\delta(1+\beta) = \alpha\lambda, \tag{77}$$

which directly yields the oscillation amplitude:

$$\delta = \frac{\alpha\lambda}{2(1+\beta)}. \tag{78}$$

The Heavy Ball method tends to bounce back and forth across coordinate boundaries, creating a zigzag motion.

Because the bouncing width 2δ depends directly on the step-size α, the coordinates can never land exactly on zero while α is fixed; to suppress these oscillations and approach true sparsity, one would ideally need to drive the step-size to zero ($\alpha \to 0$) as the optimization progresses.

2.2.4 Stopping Criteria and Practical Termination.

Classical stopping rules for unconstrained smooth optimization rely on checking whether the gradient norm drops below a prescribed tolerance threshold ($\|\nabla f(\Theta_t)\| \leq \varepsilon$). In our non-smooth and non-convex setting, this condition cannot be used:

1. **Failure of the Gradient Norm as an Optimality Proxy:** By the generalized Fermat rule of convex analysis [6, Chapter VII, Theorem 1.1.1, p. 293], an optimal solution Θ^* must satisfy the condition $0 \in \partial f(\Theta^*)$, for our composite objective this requires $0 \in \nabla\mathcal{L}(\Theta^*)_i + \lambda\partial|\Theta_i^*|$ for every coordinate i. When a parameter is zero ($\Theta_i^* = 0$), recall from the subdifferential characterization in (35) that $\partial_i(\lambda\|\Theta^*\|_1) = [-\lambda, \lambda]$, which yields:

$$\partial_i f(\Theta^*) = \nabla\mathcal{L}(\Theta^*)_i + [-\lambda, \lambda] = [\nabla\mathcal{L}(\Theta^*)_i - \lambda,\ \nabla\mathcal{L}(\Theta^*)_i + \lambda]\,. \tag{79}$$

By Fermat's stationary condition for non-smooth optimization ($0 \in \partial_i f(\Theta^*)$):

$$0 \in \nabla\mathcal{L}(\Theta^*)_i + [-\lambda, \lambda] \iff -\nabla\mathcal{L}(\Theta^*)_i \in [-\lambda, \lambda] \iff |\nabla\mathcal{L}(\Theta^*)_i| \leq \lambda. \tag{80}$$

Hence, the set-valued nature of the subdifferential at the kink can absorb and cancel a non-zero loss gradient, allowing $\Theta_i^* = 0$ to be an exact optimal minimizer.

However, the Heavy Ball algorithm does not compute the whole set $\partial f(\Theta)$, it only samples a single operational vector g_t via the sign function. Because the momentum dynamics cause coordinates to oscillate around zero without landing exactly on the kink ($\Theta_{t,i} \neq 0$), the subgradient selects the extreme values $\pm\lambda$ rather than an interior element. If k_{sparse} parameters are fluctuating across zero near a local minimum (where $\nabla\mathcal{L}(\Theta_t)_i \approx 0$), the norm of the sampled subgradient remains strictly bounded away from zero:

$$\|g_t\|_2 = \sqrt{\sum_{i=1}^{n} \left(\nabla\mathcal{L}(\Theta_t)_i + \lambda\,\text{sign}(\Theta_{t,i})\right)^2} \geq \sqrt{\sum_{i\in\text{sparse}} (\pm\lambda)^2} = \lambda\sqrt{k_{\text{sparse}}} > 0. \tag{81}$$

For instance, with $\lambda = 10^{-3}$ and $k_{\text{sparse}} = 100$ oscillating parameters, $\|g_t\|_2 \geq 10^{-2}$. Therefore, standard stopping conditions such as $\|g_t\|_2 \leq 10^{-4}$ can never be satisfied, even if the iterate is already in the immediate vicinity of an optimal point.

2. **Lack of Monotonic Descent:** Because subgradient directions are not guaranteed to be directions of descent [3, Section 5.1, p. 132] and momentum carries kinetic energy from past iterations, the Heavy Ball method does not generate monotonically decreasing objective values ($f(\Theta_{t+1}) \leq f(\Theta_t)$ may fail).

Because a mathematically certified optimality stopping criterion is unavailable for this heuristic combination of momentum and subgradients, termination must rely strictly on practical heuristics. Specifically, execution terminates when either of the following two conditions is met:

(a) **Relative Objective Stagnation:** The relative decrease between consecutive iterates falls below a prescribed tolerance threshold ε:

$$\frac{|f(\Theta_{t+1}) - f(\Theta_t)|}{\max\{1, |f(\Theta_t)|\}} < \varepsilon. \tag{82}$$

(b) **Maximum Iterations Budget:** The iteration counter reaches the computational budget $t \geq t_{\max}$.

In practice, due to the limit-cycle oscillations around the L_1 kinks identified in (78), the step-by-step progress $|f(\Theta_{t+1}) - f(\Theta_t)|$ can bounce irregularly. We therefore expect the algorithm to either exhaust the maximum iteration budget $t_{\max}$ or stop only when the trajectory becomes trapped in an oscillating regime where the overall loss progression stagnates.

2.3 Expectations for the Proximal Bundle Method ($A2$)

Based on the cutting-plane stabilization framework and the non-smooth properties of our regularized problem $f(\Theta) = \mathcal{L}(\Theta) + \lambda\|\Theta\|_1$, we formulate four core expectations regarding the computational cost per iteration, theoretical convergence guarantees, stabilization around non-smooth kinks, and stopping certificates.

2.3.1 Computational Workload and Master Problem Overhead.

The computational effort per iteration of the Proximal Bundle Method decomposes into three distinct stages:

$$\mathcal{C}_{\text{iteration}} = \mathcal{C}_{\text{eval}} + \mathcal{C}_{\text{QP}} + \mathcal{C}_{\text{recovery}}. \tag{83}$$

1. **First-Order Objective Evaluation ($\mathcal{C}_{\text{eval}}$):** Evaluating the loss and its gradient in full-batch mode requires processing all N training samples. Both the forward pass and the backpropagation sweep scale linearly with the parameter dimension n. Including the coordinate-wise evaluation of the regularizer subgradient, the evaluation cost scales as:

$$\mathcal{C}_{\text{eval}} = \mathcal{O}(N \cdot n). \tag{84}$$

   In the Bundle method, this evaluation is performed at every step, including null steps where candidate trial points are rejected as stability centers.

2. **Dual Master Problem Setup and Resolution ($\mathcal{C}_{\text{QP}}$):** Let $|\mathcal{B}|$ denote the current number of cutting planes stored in the bundle, with $|\mathcal{B}| \ll n$. Because solving the primal QP in $n+1$ variables is computationally prohibitive for high dimensions, we solve the Wolfe Dual Master Problem over the unit simplex $\Delta_{|\mathcal{B}|}$:

$$\min_{\mu \in \Delta_{|\mathcal{B}|}} \left\{ \frac{\tau}{2} \mu^T Q \mu + \alpha^T \mu \right\}, \quad \text{where } \Delta_{|\mathcal{B}|} = \left\{ \mu \in \mathbb{R}_+^{|\mathcal{B}|} : \sum_{j \in \mathcal{B}} \mu_j = 1 \right\}. \tag{85}$$

   Executing this subproblem introduces two distinct costs:

   - *Updating Q:* The matrix Q collects the pairwise dot products between the stored subgradients ($Q_{ij} = \langle g_i, g_j \rangle$). When a new subgradient cut $g^+ \in \mathbb{R}^n$ is acquired, updating this symmetric matrix requires computing $|\mathcal{B}|$ inner products between vectors of length n, which costs $\mathcal{O}(|\mathcal{B}| \cdot n)$.
   - *Solving the Dual Quadratic Program:* Standard active-set or interior-point QP solvers optimize over the $|\mathcal{B}|$ dual variables, requiring $\mathcal{O}(|\mathcal{B}|^3)$ operations per call. Because the bundle size is kept small (typically $|\mathcal{B}| \leq 50$), this subproblem executes rapidly.

3. **Primal Direction Recovery ($\mathcal{C}_{\text{recovery}}$):** Once the optimal dual multipliers $\mu^* \in \Delta_{|\mathcal{B}|}$ are obtained, the candidate parameter displacement $d^* \in \mathbb{R}^n$ is reconstructed as a weighted linear combination of the stored subgradients:

$$d^* = -\tau \sum_{j \in \mathcal{B}} \mu_j^* g_j \in \mathbb{R}^n, \tag{86}$$

   which incurs an algebraic cost of $\mathcal{O}(|\mathcal{B}| \cdot n)$ operations.

Summing these contributions, the total per-iteration time complexity is:

$$\mathcal{C}_{\text{iteration}} = \mathcal{O}(N \cdot n) + \mathcal{O}(|\mathcal{B}|^3) + \mathcal{O}(|\mathcal{B}| \cdot n). \tag{87}$$

The auxiliary memory footprint scales as $\mathcal{O}(|\mathcal{B}| \cdot n)$ to store the historical subgradient vectors. While the Heavy Ball method runs in pure $\mathcal{O}(n)$ time per update without auxiliary routines, the Bundle method incurs an internal algorithmic overhead of $\mathcal{O}(|\mathcal{B}|^3 + |\mathcal{B}| \cdot n)$ at every step. Because we deliberately adopt a compact shallow architecture ($n \sim 10^3$), the dataset pass $\mathcal{O}(N \cdot n)$ remains the dominant computational term.

2.3.2 Theoretical Convergence Results and Inapplicability.

To understand what mathematical guarantees govern the Proximal Bundle Method, we inspect the classical convergence theory developed for cutting-plane schemes:

Classical Results in Non-Smooth Convex Optimization. The theoretical convergence of the Proximal Bundle Method is established by Kiwiel [7, Chapter 3, Theorem 3.1, pp. 45–52] and Frangioni [2, Theorem 2.1, p. 88]:

- *Assumptions:* The objective function $f : \mathbb{R}^n \to \mathbb{R}$ is convex and continuous on $\mathbb{R}^n$, with bounded subgradients ($\|g\|_2 \leq M$) on bounded sets. Crucially, the sequence of proximity parameters must remain strictly bounded away from zero and infinity across all iterations:

$$0 < \tau_{\min} \leq \tau_k \leq \tau_{\max} < +\infty. \tag{88}$$

  The uniform bounds in (88) are essential: $\tau_k \geq \tau_{\min} > 0$ prevents the candidate step $d^* = -\tau_k g_{\text{agg}}$ and the predicted descent v^* from vanishing at non-stationary points, while $\tau_k \leq \tau_{\max} < +\infty$ preserves strict convexity and prevents the Master Problem from degenerating into an unbounded cutting-plane LP.

- *Consequences:* The sequence of stability centers $\{\bar{\Theta}_k\}$ generated by serious steps converges to an optimal solution Θ^* satisfying $0 \in \partial f(\Theta^*)$. Furthermore, an infinite sequence of consecutive null steps cannot stall: the parameter adaptation and cutting planes ensure that the predicted descent asymptotically vanishes:

$$\lim_{k \to \infty} \left(\frac{\tau_k}{2} \left\| \sum_{j \in \mathcal{B}_k} \mu_j^* g_j \right\|_2^2 + \sum_{j \in \mathcal{B}_k} \mu_j^* \alpha_j \right) = 0, \tag{89}$$

  thereby either triggering a serious step or certifying optimality within finite iterations.

The Closest Available Results in the Non-Convex Setting. Because our empirical loss $\mathcal{L}(\Theta)$ contains nonlinear tanh activations, the objective $f(\Theta)$ is non-convex. The closest theoretical frameworks available for non-convex bundle methods are provided by Hare and Sagastizábal [5, Theorem 1, p. 12]:

- *Assumptions:* The function f is non-convex but *weakly convex*, meaning there exists a constant $\rho \geq 0$ such that $f(\Theta) + \frac{\rho}{2}\|\Theta\|_2^2$ is convex. Furthermore, the linearization errors with respect to the current stability center $\bar{\Theta}$:

$$\alpha_j \coloneqq f(\bar{\Theta}) - f(\Theta_j) - \langle g_j, \bar{\Theta} - \Theta_j \rangle, \tag{90}$$

  must remain non-negative for all stored cuts: $\alpha_j \geq 0$.

- *Consequences:* Every cluster point of the sequence of stability centers $\{\bar{\Theta}_k\}$ is a first-order stationary point for non-smooth functions, satisfying the generalized inclusion $0 \in \partial_C f(\bar{\Theta}^*)$.

Why Classical Assumptions Break Down in Our Implementation. In our problem setting, the convergence guarantees of the classical Proximal Bundle Method cannot be applied due to the non-convex nature of the objective:

1. *Violation of Global Underestimation and Negative Linearization Errors:* In convex optimization, the subgradient inequality guarantees that the affine cut is a global lower bound ($f(\bar{\Theta}) \geq f(\Theta_j) + \langle g_j, \bar{\Theta} - \Theta_j \rangle$), which algebraically enforces $\alpha_j \geq 0$. However, because our neural loss $\mathcal{L}(\Theta)$ possesses directions of negative curvature ($\nabla^2 \mathcal{L} \not\succeq 0$), the function can curve downward. Consequently, an affine linearization generated at a distant point Θ_j can lie strictly above $f(\bar{\Theta})$, causing α_j to become strictly negative ($\alpha_j < 0$).

2. *Distortion of the Dual Master Problem and Heuristic Correction:* In the Dual Master Problem (85), the linear term is $\sum_{j \in \mathcal{B}} \mu_j \alpha_j$. If some errors are negative ($\alpha_j < 0$), this linear component becomes negative, distorting the predicted descent v^* and invalidating the theoretical stopping certificate. To maintain numerical stability in our solver, we enforce non-negativity via:
$$\tilde{\alpha}_j = \max\{0, \alpha_j\}. \tag{91}$$
While this adjustment allows the QP solver to run without issues, modifying the cuts means the cutting-plane model is no longer a certified lower bound, breaking the assumptions of the classical convergence proofs.

2.3.3 Trajectory Behavior and Kink Stabilization.

The defining practical advantage of the Proximal Bundle Method over the Heavy Ball method lies in how it handles the non-differentiable coordinate kinks $\Theta_i = 0$ introduced by the Lasso penalty $\lambda \|\Theta\|_1$.

Recall from (86) that the search direction is computed as an aggregate combination of stored subgradients:
$$d^* = -\tau \, g_{\text{agg}}, \quad \text{where} \quad g_{\text{agg}} = \sum_{j \in \mathcal{B}} \mu_j^* g_j, \quad \mu^* \in \Delta_{|\mathcal{B}|}. \tag{92}$$

Consider an iterate near a non-differentiable boundary where a coordinate crosses zero ($\Theta_i \approx 0$):

- When a candidate step overshoots a kink, the newly evaluated subgradient g^+ jumps abruptly by 2λ, reversing its sign along coordinate i.

- In the Bundle Method, the trial point is rejected as a stability center if the objective value does not improve sufficiently. The opposing subgradient g^+ is then stored in the memory bundle $\mathcal{B}$.

At the next iteration, the bundle contains cuts sampled on both sides of the zero boundary: planes evaluated at positive values ($\Theta_{j,i} > 0$) have a regularizer subgradient of $+\lambda$, while those evaluated at negative values ($\Theta_{j,i} < 0$) have $-\lambda$. The dual QP solver can then assign positive weights $\mu_1^*, \mu_2^* > 0$ to these opposing cuts:
$$(g_{\text{agg}})_i = \mu_1^*(+\lambda) + \mu_2^*(-\lambda) = \lambda(\mu_1^* - \mu_2^*). \tag{93}$$

In the idealized symmetric case where the smooth gradient is negligible ($\nabla\mathcal{L}(\Theta^*)_i \approx 0$), the dual solver can assign equal weights $\mu_1^* \approx \mu_2^* \approx 0.5$, causing the aggregate component to vanish: $(g_{\text{agg}})_i \approx 0$. More generally, the QP solver adjusts the multipliers such that $\mu_1^* - \mu_2^* \approx -\nabla\mathcal{L}(\Theta^*)_i/\lambda$, which neutralizes the directional thrust whenever $|\nabla\mathcal{L}(\Theta^*)_i| \leq \lambda$.

Geometrically, the cutting planes form a V-shape that guides the next step directly onto the kink at $\Theta_i = 0$. Consequently, this mechanism should allow the Proximal Bundle Method to stabilize coordinates directly on the non-smooth interface $\Theta_i = 0$, thereby yielding exact parameter sparsity and avoiding the persistent zigzag oscillations typical of the Heavy Ball method.

2.3.4 Stopping Criteria and Certified Optimality Gap.

A major advantage of the Proximal Bundle Method over the Heavy Ball heuristic is that it natively provides a reliable certificate of near-optimality through the dual Master Problem.

Solving the dual QP (85) yields the optimal predicted descent:

$$-v^* = \tau\|g_{\text{agg}}\|_2^2 + \alpha_{\text{agg}} \geq 0 \tag{94}$$

where $g_{\text{agg}} = \sum_{j\in\mathcal{B}} \mu_j^* g_j$ and $\alpha_{\text{agg}} = \sum_{j\in\mathcal{B}} \mu_j^* \alpha_j$.

The quantity $-v^*$ measures the maximum descent predicted by the cutting-plane model: if $-v^* \approx 0$, then necessarily $\|g_{\text{agg}}\|_2 \approx 0$ and $\alpha_{\text{agg}} \approx 0$.

Because the dual Master Problem (51) restricts the multipliers to the unit simplex ($\mu^* \geq 0$ and $\sum_{j\in\mathcal{B}} \mu_j^* = 1$), the aggregate vector $g_{\text{agg}} = \sum_{j\in\mathcal{B}} \mu_j^* g_j$ is a convex combination of accumulated subgradients.

By the convexity of the subdifferential $\partial f(\bar{\Theta})$, any convex combination of valid subgradients remains a valid subgradient [6], consequently when $\|g_{\text{agg}}\|_2 \approx 0 \implies g_{\text{agg}} \approx 0$, it directly follows that $g_{\text{agg}} = 0 \in \partial f(\bar{\Theta})$, certifying that the current stability center $\bar{\Theta}$ is close to a stationary point.

In our implementation, execution terminates under either of two conditions:

(a) **Model Optimality Certificate:** The predicted descent satisfies $-v^* \leq \varepsilon_{\text{bundle}}$.

(b) **Computational Budget Exhaustion:** The total count of objective evaluations reaches $t \geq t_{\max}$.

2.4 Analytical Complexity and the Impact of Large Dimension n

We conduct this complexity comparison to motivate why, in our experimental setup, the network parameter dimension is deliberately kept moderate at $n \approx 10^3$.

- **Primal Master Problem Bottleneck:** In the primal space, the QP solver must optimize over the parameter displacement $d \in \mathbb{R}^n$, which involves $n+1$ decision variables. Standard quadratic programming algorithms incur a computational cost that scales cubically with the number of variables[1, Chapter 11, Section 11.8, pp. 615–618]:

$$\mathcal{C}_{\text{primal}}^{\text{QP}} = \mathcal{O}\left((n+1)^3\right) \approx \mathcal{O}\left(n^3\right). \tag{95}$$

  In high-dimensional machine learning regimes (e.g., $n = 10^5$ or $n = 10^6$), solving a problem with this cubic dependence requires $\mathcal{O}(10^{15}) - \mathcal{O}(10^{18})$ operations per step, rendering the primal approach completely intractable.

- **Dual Master Problem Scaling:** Solving the Wolfe Dual shifts the optimization onto the simplex $\Delta_{|\mathcal{B}|}$ over $|\mathcal{B}|$ multipliers, decoupling the QP solver's nominal variable count from n ($|\mathcal{B}| \ll n$). However, the linear dependence on n persists through the Gram matrix updates and the primal displacement reconstruction:

$$\mathcal{C}_{\text{dual}}^{\text{total}} = \mathcal{O}\left(|\mathcal{B}|^3\right) + \mathcal{O}(|\mathcal{B}| \cdot n) = \mathcal{O}\left(|\mathcal{B}|^3 + |\mathcal{B}| \cdot n\right). \tag{96}$$

To concretely illustrate this divergence across varying problem sizes, Table 1 compares the theoretical floating-point operations per step for a typical bundle memory of $|\mathcal{B}| = 30$ cutting planes.

Dimension n	**Primal QP** $\mathcal{O}(n^3)$	**Dual QP** $\mathcal{O}(\|\mathcal{B}\|^3)$	**Dual Overhead** $\mathcal{O}(\|\mathcal{B}\|n)$	**Heavy Ball** $\mathcal{O}(n)$
10^2	10^6 ops	2.7×10^4 ops	3.0×10^3 ops	10^2 ops
10^3	10^9 ops	2.7×10^4 ops	3.0×10^4 ops	10^3 ops
10^4	10^{12} ops	2.7×10^4 ops	3.0×10^5 ops	10^4 ops
10^5	10^{15} ops	2.7×10^4 ops	3.0×10^6 ops	10^5 ops
10^6	10^{18} ops	2.7×10^4 ops	3.0×10^7 ops	10^6 ops

Table 1: Theoretical operation scaling per iteration assuming a bundle size $|\mathcal{B}| = 30$.

The numerical breakdown in Table 1 highlights a clear structural trade-off: moving to the dual formulation successfully removes the catastrophic cubic cost $\mathcal{O}(n^3)$, keeping the internal QP solve fixed at a negligible 2.7×10^4 operations regardless of n. However, because each cutting plane stores a full subgradient vector, manipulating the bundle still requires computing inner products of dimension n. For deep architectures with millions of weights, this overhead generates tens of millions of operations per iteration, making the method scale poorly. On the contrary, the Heavy Ball method executes each update through cheap coordinate-wise vector operations that scale effortlessly as $\mathcal{O}(n)$.

References

[1] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, Cambridge, 2004.

[2] Antonio Frangioni. Standard bundle methods: Untrusted models and duality. In Adil M. Bagirov, Manlio Gaudioso, Napsu Karmitsa, Marko Mäkelä, and Sona Taheri, editors, *Numerical Nonsmooth Optimization: State of the Art Algorithms*, pages 61–116. Springer, Cham, 2020.

[3] Antonio Frangioni. Optimization & Learning: Lecture notes. Lecture notes, Department of Computer Science, University of Pisa, 2025.

[4] Euhanna Ghadimi, Hamid Reza Feyzmahdavian, and Mikael Johansson. Global convergence of the Heavy-ball method for convex optimization. In *2015 European Control Conference (ECC)*, pages 310–315, 2015.

[5] Warren Hare and Claudia Sagastizábal. A redistributed proximal bundle method for nonconvex optimization. Technical report, IRMACS, Simon Fraser University / CEPEL, 2009. Preprint.

[6] Jean-Baptiste Hiriart-Urruty and Claude Lemaréchal. *Convex Analysis and Minimization Algorithms I: Fundamentals*, volume 305 of *Grundlehren der mathematischen Wissenschaften*. Springer-Verlag, 1993.

[7] Krzysztof C. Kiwiel. Proximity control in bundle methods for convex nondifferentiable minimization. *Mathematical Programming*, 46(1-3):105–122, 1990.

[8] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering. Springer, New York, 2nd edition, 2006.

[9] Peter Ochs. Local convergence of the Heavy-Ball method and iPiano for non-convex optimization. *Journal of Optimization Theory and Applications*, 177:153–180, 2018.

[10] Peter Ochs, Yunjin Chen, Thomas Brox, and Thomas Pock. iPiano: Inertial proximal algorithm for nonconvex optimization. *SIAM Journal on Imaging Sciences*, 7(2):1388–1419, 2014.

[11] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[12] Walter Rudin. *Principles of Mathematical Analysis*. International Series in Pure and Applied Mathematics. McGraw-Hill, New York, 3rd edition, 1976.